

Computer Systems Preparatory Code

Important lines

\$args – commandline arguments

\$lines=Get-Content \$args[0] – reads content of a file

Out-File -Append even.txt – appends output to a file

Test-Path \$args[0] -PathType Leaf)) Checks path of a file

Test-Path \$dir -PathType Container – checks path of a folder

Test-Path \$file.FullName -PathType Leaf – returns the file as an object

Even and odd lines

\$lines=Get-Content \$args[0] # Reads lines of a file passed as a commandline argument

for([int] \$i = 0; \$i -lt \$lines.Length; \$i++) # Loops through every line of the input file

```
{
    if($i % 2 -eq 0) #Checks if the line is even
    {
        $lines[$i] | Out-File -Append even.txt # appends even lines to even.txt
    }
    else
    {
        $lines[$i] | Out-File -Append odd.txt #appends odd lines to odd.txt
    }
}
```

}
\$args = comandline arguments

\$len = \$args.Length # Gets the length of commandline args

\$sum=0 #initializes sum to 0

for([int]\$i=0;\$i -lt \$len; \$i++) # loops through the commandline-args

```
{
    $sum=$sum+$args[$i] #Sums the commandline args
}
```

Write-Output "Sum of input arguments:\$sum" #Outputs the sum

Switch Case

\$a=Read-Host "Write your fav fruit:"

```
switch($a)
{
    "apple" {"a value:"+$a}
```

```

"orange" {"a value:"+$a}
"mango" {"a value:"+$a}
default {"a value is unknown:"+$a}
}

```

Taking a path to a file and using it to run

if ((\$args.Length -gt 0) -and (Test-Path \$args[0] -PathType Leaf)) # checks if the argument is a file , container its checking for a folder

```

{
    $contents= Get-Content $args[0]

```

```

}

```

pipeline

#input means you receive sth from the pipeline

```
$Get-Content .\emails.txt | .\email.ps1
```

else

```

{
    $contents=$input # gets the input content if the file is missing
}

```

#for loop, you can't get the length of the pipeline

foreach(\$line in \$contents) #foreach iterates from the start to the end

```

{
    Write-Output $line.Replace("@", " at ").Replace(".", " dot ")
}

```

File Summary

```
Param(
```

```
    [Parameter(Mandatory=$true)]$directory
```

```
)
```

#function that calculates the size of files in a directory

```
function GetDirectorSize($dir)
```

```

{
    $size=0
    if(Test-Path $dir -PathType Container) #Checks if the path is a folder
    {
        $length = Get-ChildItem $dir #gets all the files in a folder
        foreach($file in $length) #loops through each file in the folder
        {
            if (Test-Path $file.FullName -PathType Leaf) #checks if path is a file
            {
                $size+=$file.Length #accumulates the size of the files
            }
        }
    }
}

```

```

    }
    else
    {
        $size+=GetDirectorSize($file) # recursively calls the function for subfolders
    }
}
}

return $size
}

```

Write-Output (GetDirectorSize(\$directory)) #calling the function

Gcd function

```

$i = [System.Math]:: Min([int]$args[0], [int]$args[1])

while ($i -gt 1 -and [int]$args[0] % $i -ne 0 -or [int]$args[1] % $i -ne 0)
{
    $i--
}

Write-Output $i

```

Pipelining

```

$name=Read-Host "Enter name of the user to count"
Write-Host "Counting" $name
(Get-Content .\pass.txt |
    ForEach-Object { $_.Split(':')[4]} |
    Select-String $name |
    Measure-Object).Count

Get-Content $args[0] |
    ForEach-Object { $_.Split(' ')[0]} |
    Where-Object { $_.Length -gt $n} |
    ForEach-Object {Write-Output $_}

```